

Autoscaling gerarchico

Modellazione, simulazione e valutazione di un'infrastruttura cloud

Francesco Masci

Università degli Studi di Roma Tor Vergata | A.A. 2025/2026

Agenda

01 Sistema e obiettivi

Architettura e domanda di ricerca.

02 Modellazione

Modello concettuale e specifiche.

03 Implementazione

Modello computazionale.

04 Verifica e validazione

Tracce deterministiche e confronto con JMT.

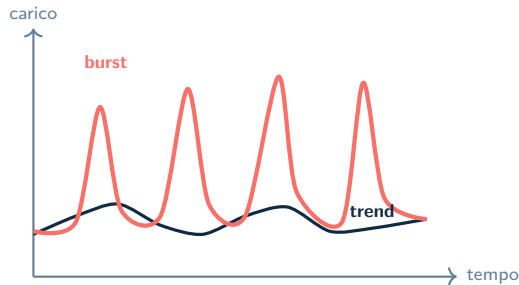
05 Risultati

Dimensionamento, transitorio e burstiness.

06 Miglioramento

Meccanismo a bande e conclusioni.

Il problema: due scale temporali, una sola infrastruttura



LUNGO PERIODO

Lo scaling orizzontale segue il trend, ma paga il tempo di boot.

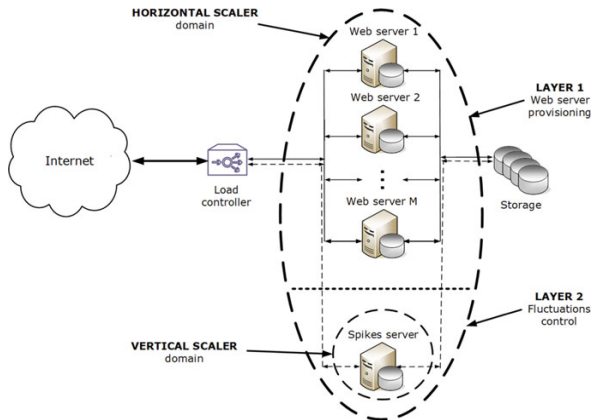
BREVE PERIODO

I burst saturano i nodi prima che una nuova istanza sia disponibile.

Domanda di ricerca

Come rispettare uno SLA di **6 s** senza ricorrere all'over-provisioning?

La risposta: elasticità su due livelli



Architettura adattata dal caso di studio in Serazzi, cap. 6.

LAYER 1 Web Server pool

- ▶ traffico ordinario;
- ▶ disciplina Processor Sharing;
- ▶ scaling orizzontale lento.

LAYER 2 Spike Server

- ▶ percorso ausiliario sempre disponibile;
- ▶ assorbe la congestione temporanea;
- ▶ scaling verticale immediato.

Il Load Controller sceglie il nodo e decide **evento per evento** se assegnarli il job o deviarlo.

Obiettivi e percorso sperimentale

1. Dimensionare

Routing, soglia di deviazione, incremento verticale e soglie dello scaling orizzontale.

2. Valutare

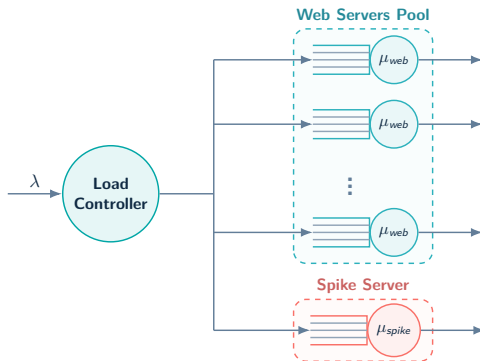
Trade-off costo–latenza, transitorio e robustezza alla burstiness.

3. Migliorare

Ridurre il chattering del router con un controllo a bande.



Concettuale I : una rete di code gerarchica



$N(t)$ STAZIONI PARALLELE

1 STAZIONE AUSILIARIA

PROCESSOR SHARING

Concettuale II : lo stato del sistema

$$S(t) = \left\{ N(t), \mathbf{n}(t), n_s(t), v_s(t), \widehat{W}_{mw}(t), \tau_h(t), \tau_v(t) \right\}$$

STATO FISICO

- ▶ $N(t)$: Web Server attivi;
- ▶ $\mathbf{n}(t) = (n_1, \dots, n_{N(t)})$: carico del pool;
- ▶ $n_s(t)$: job sullo Spike Server;
- ▶ $v_s(t)$: capacità istantanea dello Spike.

MEMORIA DEL CONTROLLO

- ▶ $\widehat{W}_{mw}(t)$: risposta media nella moving window;
- ▶ $\tau_h(t)$: cooldown orizzontale residuo;
- ▶ $\tau_v(t)$: cooldown verticale residuo.

Transizioni

Arrivo, completamento e monitoraggio periodico.

Specifiche I : workload e disciplina di servizio

PROCESSO DI ARRIVO

$$T_j = A_j - A_{j-1}, \quad E[T] = \frac{1}{\lambda}$$

$$T \sim \begin{cases} \text{Exp}(m_1) & \text{con probabilità } p \\ \text{Exp}(m_2) & \text{con probabilità } 1 - p \end{cases}$$

- ▶ H_2 per rappresentare $C_V^A > 1$;
- ▶ $\lambda \in [2, 14]$ job/s;
- ▶ burstiness: $C_V^A \in \{1, 4, 8, 12\}$.

PROCESSO DI SERVIZIO

$$S_j \sim H_2 = \begin{cases} \text{Exp}(\nu_1) & \text{con probabilità } q \\ \text{Exp}(\nu_2) & \text{con probabilità } 1 - q \end{cases}$$

- ▶ S_j è la domanda di servizio del job j ;
- ▶ $\mu_{web} = \mu_{base} = 4.0$ job/s
- ▶ anche il servizio è iperesponenziale: $C_V^S = 4.0$.

DISCIPLINA PROCESSOR SHARING

$$\mu_i^{job}(t) = \frac{\mu_i(t)}{n_i(t)}, \quad \mu_i(t) = \mu_{web}$$

$$\mu_s^{job}(t) = \frac{\nu_s(t)\mu_{base}}{n_s(t)}$$

Servizio immediato e capacità equamente ripartita.

Specifiche II : parametri fissati del controllo

LEGAMI CON $SI_{\max}$

$$W = 2.5 SI_{\max}$$

$$L_{sv}^{\max} = \frac{SI_{\max}}{2.5}, \quad L_{sv}^{\min} = 0.7 L_{sv}^{\max}$$

La finestra orizzontale e le soglie dello scaler verticale restano proporzionate alla congestione ammessa sul Web Server.

Con $SI_{\max} = 40$

$$W = 100$$

completamenti nella moving window

$$L_{sv}^{\max} = 16$$

soglia di Speed Up

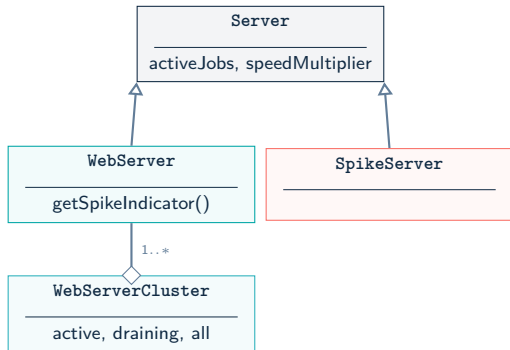
$$L_{sv}^{\min} = 11.2$$

soglia di Speed Down

VALORI FISSATI A PRIORI

non appartengono alla successiva campagna di tuning

Computazionale I : oggetti di dominio e Processor Sharing



```
double work = elapsed * speedMultiplier / activeJobs.size();
for (Job job : activeJobs) {
    job.decreaseRemainingDemand(work);
}
```

NUCLEO PS

$$\Delta w_j = \frac{s \Delta t}{|\text{activeJobs}|}$$

- ▶ ogni job entra subito in servizio;
- ▶ la domanda residua viene aggiornata a ogni evento;
- ▶ lo scale-in sposta il server in *draining*;
- ▶ i job già assegnati terminano senza perdite.

Draining Server

Un server rimosso non riceve nuovi job, ma resta operativo finché non si svuota.

Computazionale II : motore next-event

1. Estrai il primo evento dalla PriorityQueue

2. Avanza il lavoro di tutti i job fino a t_{next}

3. Aggiorna clock e stato

4. Gestisci arrivo, completamento o monitoraggio

5. Rischeda i completamenti interessati

```
if (eventQueue.isEmpty()) return false;

Event event = eventQueue.poll();
double nextTime = event.getTime();

processActiveJobs(nextTime - clock, nextTime);
clock = nextTime;
processEvent(event);
return true;
```

FUTURE EVENT LIST

$$t_j^{comp} = t + \frac{d_j n(t)}{s}$$

- ▶ arrivo: cambia $n(t)$;
- ▶ completamento: cambia $n(t)$;
- ▶ speed up/down: cambia s ;
- ▶ ogni variazione modifica le date future.

Il clock salta direttamente al prossimo evento: nessun passo temporale artificiale.

Computazionale III : Batch Means e repliche indipendenti

Builder

SimulationBuilder
Componenti dalla
configurazione.

Facade

SimulationFacade
Batch Means e repliche
indipendenti.

Collector

Decorator
Serie, checkpoint, CSV e
JSON.

BATCH MEANS

```
Simulator controller = builder.build();

if (warmUpJobs > 0) {
    controller.run(untilJobsCompleted(warmUpJobs));
}

controller.resetStatistics();

for (int i = 0; i < numBatches; i++) {
    controller.run(untilJobsCompleted(batchSize));
    updateAggregators(controller);
    controller.resetStatistics();
}
```

Un solo sistema evolve nel tempo; ogni batch produce un'osservazione dopo il warm-up.

REPLICHE INDIPENDENTI

```
long currentSeed = config.execution().seed();

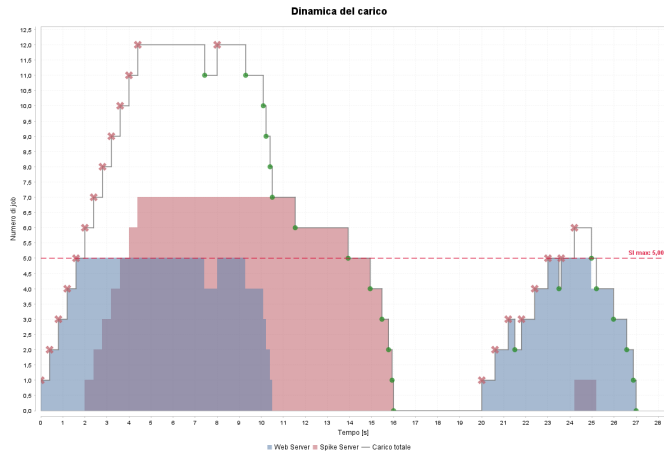
for (int i = 0; i < numReplications; i++) {
    Simulator controller = new SimulationBuilder()
        .config(config.withSeed(currentSeed))
        .build();

    controller.run(untilTimeElapsed(maxTime));
    updateAggregators(controller);

    // Stream successivo, non riuso lo stesso seed
    currentSeed = controller.getSeed();
}
```

Ogni replica ricostruisce il sistema; il seed finale della run diventa il seed iniziale della successiva.

Verifica: il routing segue esattamente la soglia



Job sul Web Server e deviazioni verso lo Spike Server.

TRACE-DRIVEN

- ▶ arrivi noti;
- ▶ servizi noti;
- ▶ soglia nota;
- ▶ destinazione attesa nota.

21

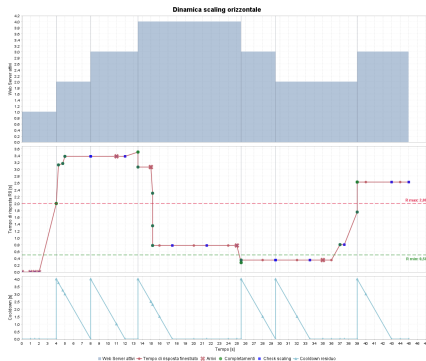
job totali

8

job deviati

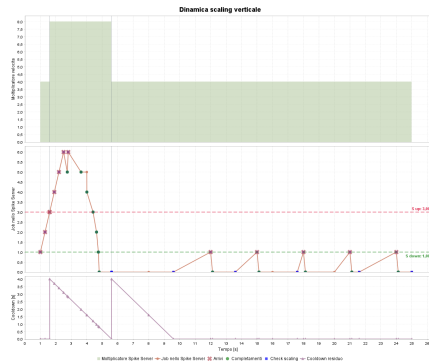
Nessuna perdita e nessun job
già assegnato viene interrotto.

Verifica: gli scaler rispettano soglie e cooldown



SCALING ORIZZONTALE

Moving window, scale out/in e blocco nel cooldown.

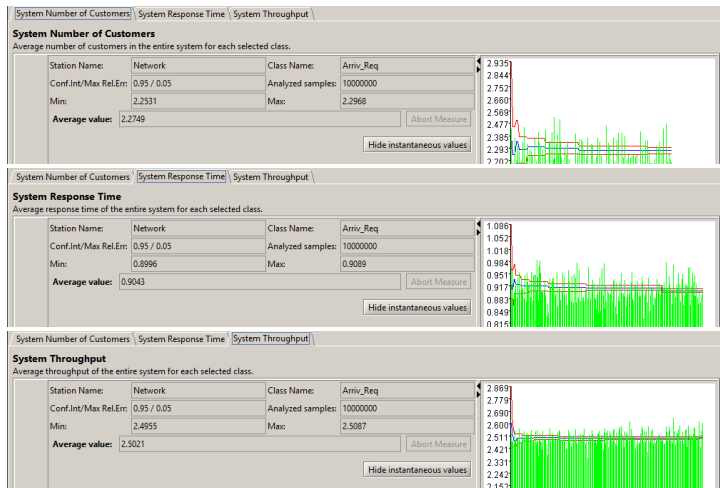


SCALING VERTICALE

Speed up a $t = 1.6$ s, speed down a $t = 5.6$ s.

Le transizioni osservate coincidono con gli istanti previsti dalle tracce deterministiche.

Validazione esterna: simulatore Java vs JMT



JOB MEDI

0.46%

scarto sui job medi

TEMPO DI RISPOSTA

0.50%

scarto su R_0

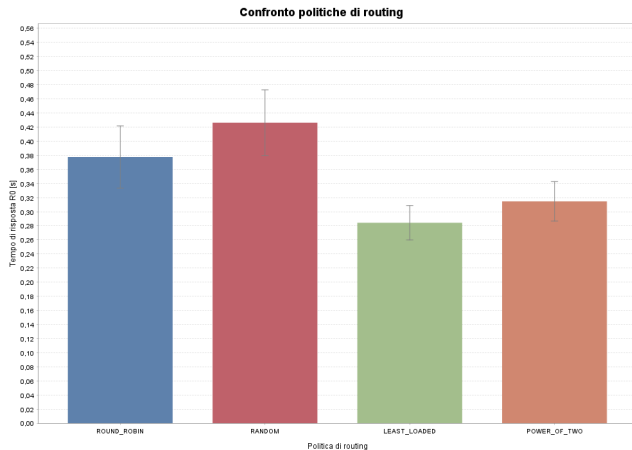
THROUGHPUT

0.34%

scarto sul throughput

Stesso workload e stessa disciplina; gli intervalli di confidenza si sovrappongono.

Obiettivo 1A: scegliere la politica di routing



SCELTA

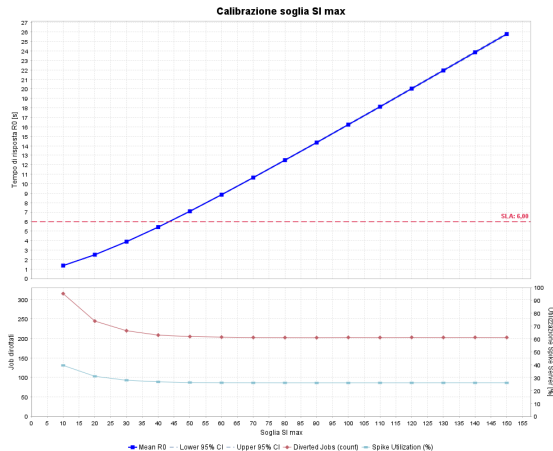
Least Loaded

- ▶ $R_0 = 0.2843$ s;
- ▶ -24.7% vs Round Robin;
- ▶ -33.3% vs Random;
- ▶ minima dispersione:
 $\sigma = 0.0243$ s.

Osservare il carico istantaneo evita micro-congestioni che il Processor Sharing, da solo, non elimina.

4 Web Server, H_2/H_1 , $\lambda = 5$, $C_V = 4$.

Obiettivo 1B: calibrare la soglia di deviazione



SCELTA

$$SI_{\max} = 40$$

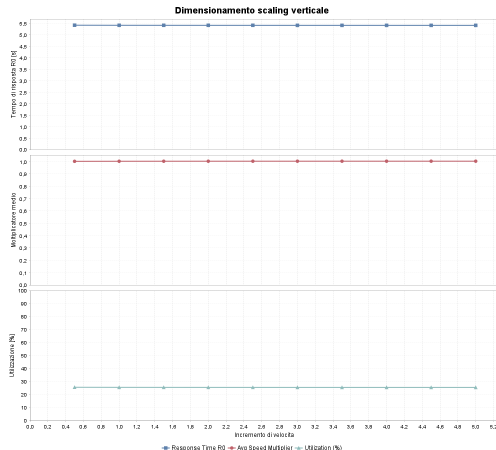
- ▶ $R_0 = 5.4209$ s;
- ▶ CI [5.3970, 5.4448];
- ▶ job devianti: 20.34%;
- ▶ utilizzo Spike: 26.77%.

Il punto di rottura

A $SI_{\max} = 50$: $R_0 = 7.0903$ s.
SLA violato.

Un Web Server, Spike Server attivo, H_2/H_2 , scaling disabilitato.

Obiettivo 1C: dimensionare lo scaling verticale



SCELTA

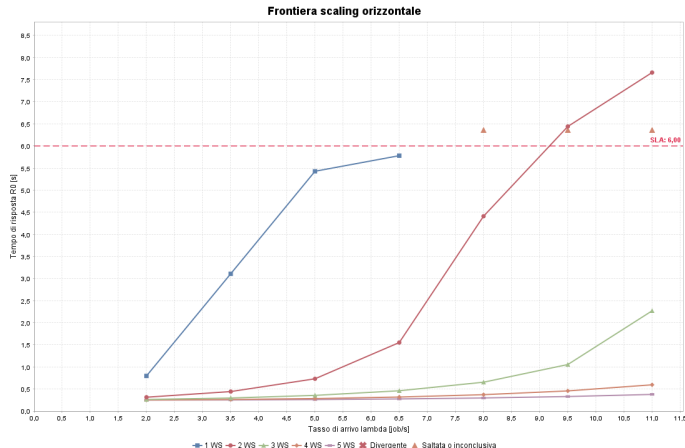
$$\Delta v_s = 1.0$$

- ▶ R_0 resta tra 5.4046 e 5.4109 s;
- ▶ tutti i CI al 95% si sovrappongono;
- ▶ più capacità non produce un beneficio misurabile.

La capacità passa da 1.0 a 2.0:
il raddoppio più sobrio, senza
sovradimensionamento.

$S_{\text{max}} = 40$; incremento additivo di capacità variato da 0.5 a 5.0.

Obiettivo 1D: tracciare la frontiera orizzontale



Frontiera statica di capacità; $SLA_{R_0} \leq 6$ s.

SCELTA NOMINALE

$$R^{\min} = 3.5 \text{ s}$$

- ▶ $R^{\max} = 6$ s: coincide con lo SLA;
- ▶ a $\lambda = 5$, 1 WS: 5.4271 s;
- ▶ a $\lambda = 11$, 3 WS: 2.2695 s;
- ▶ a $\lambda = 11$, 2 WS: 7.6609 s.

Per workload persistentemente aggressivi conviene $R^{\min} \in [1.5, 2.0]$ s.

Dimensionamento completato: la baseline



Osservazione

Finestra mobile: 100
completamenti.

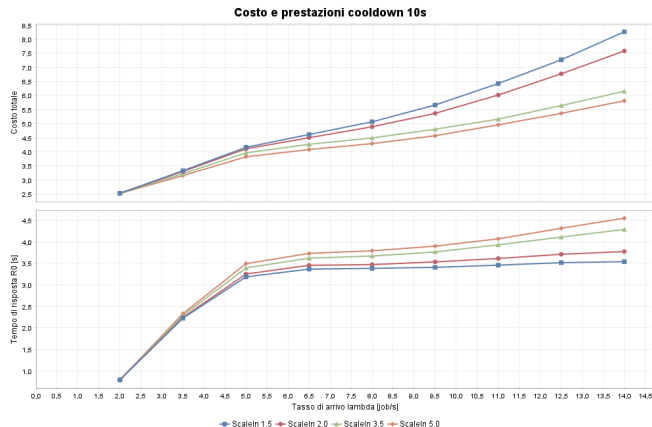
Reazione

Cooldown iniziale: 10 s.

Vincolo

SLA: $R_0 \leq 6$ s.

Costo: il cooldown conta



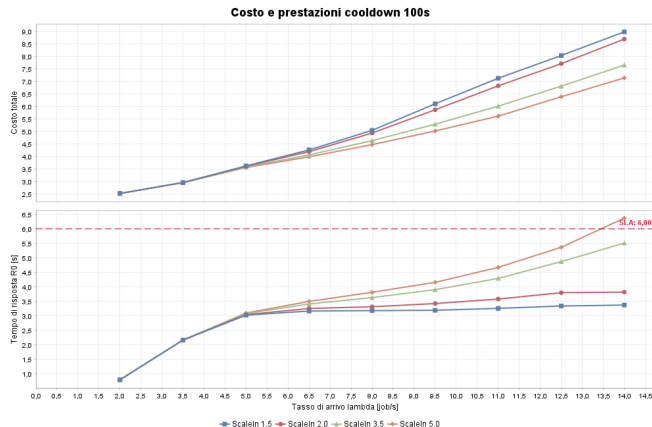
Costo aggregato e R_0 per diverse soglie di Scale In, $T_c = 10$ s.

Scelta equilibrata

- $R^{\min} = 3.5$ s;
- cooldown = 10 s;
- costo medio = 4.469;
- SLA rispettato fino a $\lambda = 11$.

Soglie più basse riducono la latenza ma mantengono più server attivi. Un cooldown da 100 s reagisce troppo lentamente: nel caso peggiore $R_0 = 6.3678$ s.

Costo: quando il controller diventa troppo lento



Costo e R_0 con cooldown 100 s.

CASO CRITICO

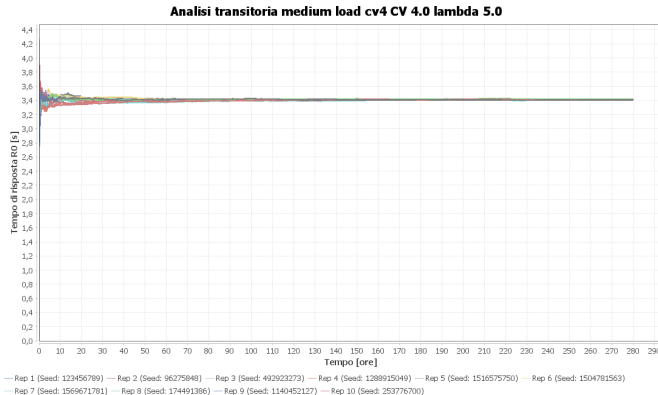
$$R_0 = 6.3678 \text{ s}$$

- $R^{\min} = 5.0 \text{ s}$;
- carico massimo;
- reazione tardiva;
- SLA violato.

Conclusione

10 s evita sia l'eccesso di riconfigurazioni del caso 5 s, sia l'inerzia dei casi 50–100 s.

Analisi del transitorio: il sistema si assesta



Scenario medio: $\lambda = 5$ job/s, $C_V = 4$, sistema inizialmente vuoto.

Cosa osserviamo

- ▶ nessuna deriva iniziale persistente;
- ▶ medie cumulative stabili;
- ▶ R_0 sotto SLA in tutti i 6 scenari;
- ▶ ai carichi medio-alti cooperano entrambi i livelli.

Interpretazione

Il transitorio non invalida le conclusioni ottenute a regime.

Transitorio: sei scenari, stesso esito

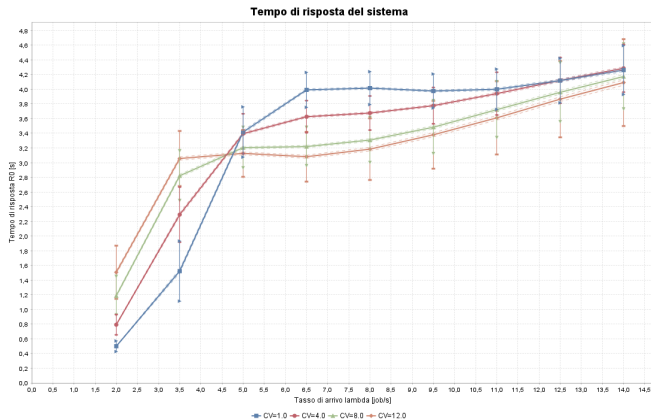
Scenario	λ	C_V	R_0 [s]	Spike
Low CV2	2.5	2	0.8102	0.00%
Low CV4	2.5	4	1.1721	0.00%
Medium CV4	5.0	4	3.4112	9.97%
Medium CV6	5.0	6	3.3106	14.72%
High CV4	8.0	4	3.6842	40.28%
High CV6	8.0	6	3.4626	43.07%

6 SU 6 SOTTO SLA

- ▶ a basso carico lo Spike resta spento;
- ▶ a carico medio assorbe i burst;
- ▶ ad alto carico supera il 40% di utilizzo;
- ▶ nessuna deriva tra le repliche.

Il sistema cambia *come* usa le risorse, non perde stabilità.

Burstiness: la gerarchia redistribuisce il carico



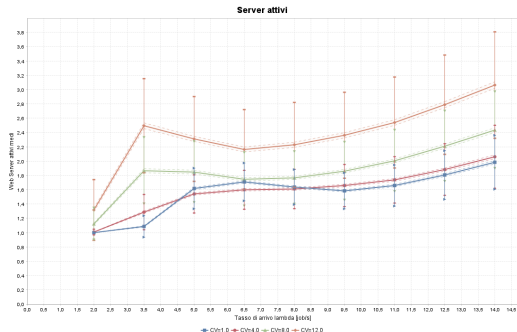
4.2853 s

massimo R_0 osservato

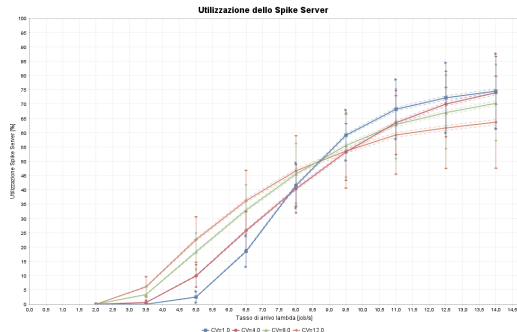
- ▶ tutta la griglia resta sotto SLA;
- ▶ a $\lambda = 5$, i burst attivano entrambi gli scaler;
- ▶ a $\lambda = 11$, il pool ordinario cresce e alleggerisce lo Spike Server.

Per $\lambda = 11$, passando da $C_V = 1$ a 12, l'utilizzazione Spike scende dal 68.18% al 59.19%.

Burstiness: chi assorbe davvero il carico?



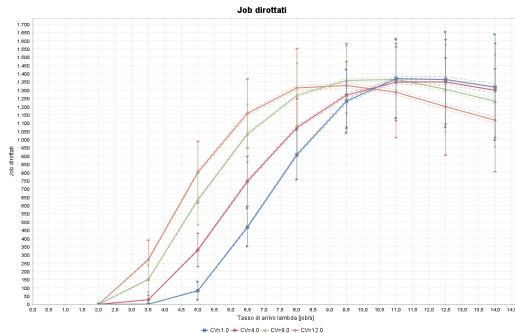
WEB SERVER ATTIVI



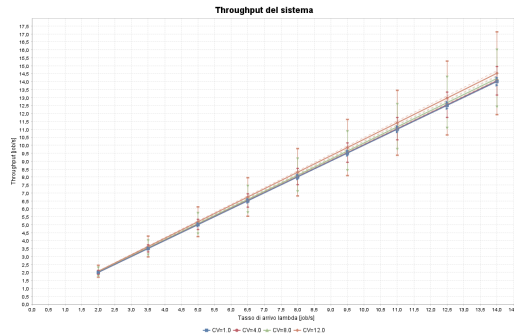
UTILIZZO SPIKE

A $\lambda = 5$ crescono entrambi. A $\lambda = 11$, il pool ordinario più ampio riduce il ricorso proporzionale allo Spike Server.

Burstiness: il controllo reagisce, il flusso resta stabile



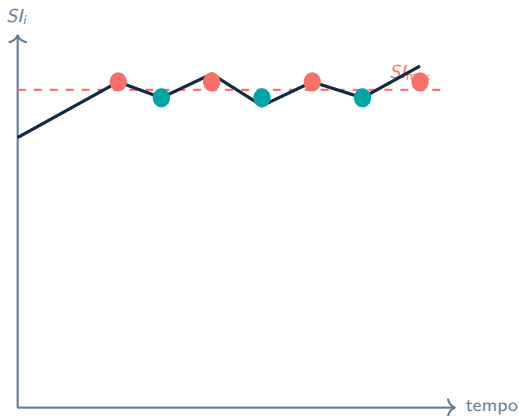
JOB DEVIATI



THROUGHPUT

Più variabilità produce più interventi e più deviazioni, ma non perdita di throughput né instabilità delle metriche.

Limite del modello base: chattering vicino alla soglia



Con una soglia rigida, piccole oscillazioni producono continui cambi di destinazione.

Miglioramento: banda

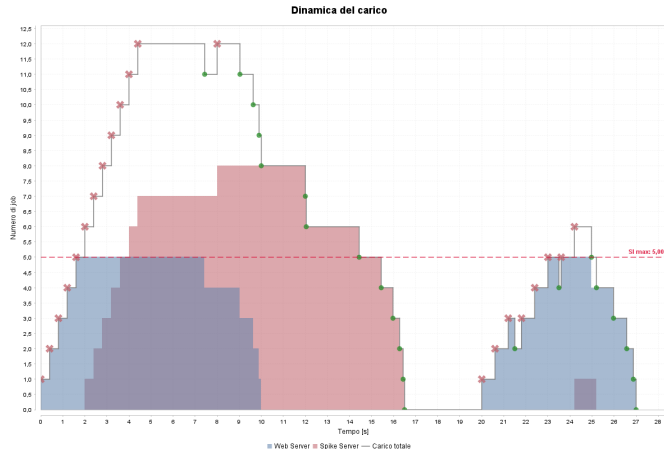
Ogni Web Server mantiene uno stato:

$$B = S_{l_{\max}} - S_{l_{\min}}.$$

Si entra in modalità spike a $S_{l_{\max}}$, ma si rientra nel percorso ordinario solo a $S_{l_{\min}}$.

La memoria stabilizza il controllo, al prezzo di una maggiore permanenza sullo Spike Server.

Meccanismo a bande: verifica trace-driven



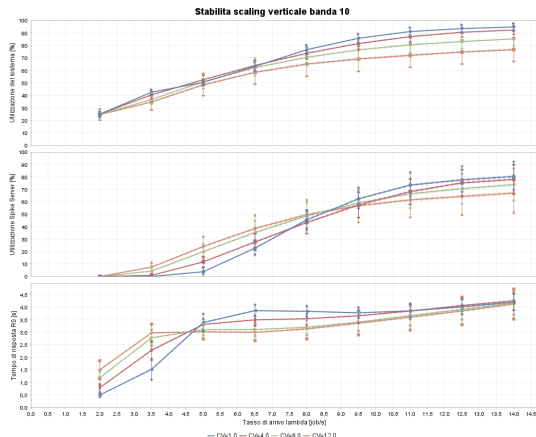
$Sl_{max} = 5$, $Sl_{low} = 2$, 21 arrivi deterministici.

COMPORTAMENTO ATTESO

- ▶ attivazione a Sl_{max} ;
- ▶ persistenza nella banda;
- ▶ rilascio solo a Sl_{low} ;
- ▶ nuova attivazione corretta.

Nessun rientro prematuro,
nessun job perso.

Sistema modificato: meno latenza, più uso dello spike



Banda $B = 10$: utilizzazioni e tempo di risposta al variare del carico.

	$B = 0$	$B = 10$
$R_0, \lambda = 5$	3.3939	3.3116
Spike util.	9.91%	11.73%
$R_0, \lambda = 11$	3.9401	3.8465
Spike util.	63.47%	68.21%

Scelta finale

$B = 10$ è il compromesso: a $\lambda = 11$ riduce la latenza del 2.4% con +4.7 punti di utilizzazione Spike.

Ampiezza della banda: quattro punti sul trade-off

B	$R_0 \lambda = 5$	Spike	$R_0 \lambda = 11$	Spike
0	3.3939	9.91%	3.9401	63.47%
5	3.3599	10.65%	3.8993	65.46%
10	3.3116	11.73%	3.8465	68.21%
20	3.1766	13.54%	3.7576	74.46%

Gli intervalli delle configurazioni estreme non si sovrappongono: il beneficio non è rumore statistico.

B=20

Latenza minima, ma +11 punti di utilizzo Spike a $\lambda = 11$.

B=10

Riduzione del 2.4% di R_0 , con +4.7 punti di utilizzo.

Scelta

$B = 10$: miglior compromesso prestazioni–costo.

Conclusioni

- ① **Due scale temporali richiedono due forme di elasticità.** Lo scaling orizzontale segue il trend; lo Spike Server assorbe i burst.
- ② **La simulazione rende esplicito il trade-off.**
 $SI_{\max} = 40$, $R^{\min} = 3.5\text{ s}$ e cooldown 10 s rispettano lo SLA senza sovradimensionare il sistema.
- ③ **La memoria migliora la stabilità.** Una banda $B = 10$ riduce il chattering e la latenza con un costo operativo controllato.



Grazie per l'attenzione